第 19 章：asyncio、coroutine 与事件循环

一句话先讲透：**asyncio 不是“自动多线程”，而是让一条任务在等待 I/O 时主动暂停，由事件循环去推进其他已经就绪的任务。**

Agent 服务经常需要等待 LLM、MCP tool、HTTP 接口或数据库返回。等待期间，当前请求没有可继续执行的 Python 代码。如果把这些等待写成真正的异步操作，一个线程就能交错推进许多请求，而不必让每个请求各占一个线程干等。

本章要建立的核心心智模型是：

```
async def 定义可暂停的函数
        ↓ 调用
coroutine 对象保存一次调用的参数和执行状态
        ↓ 被 await、Task 或 asyncio.run 驱动
函数体开始执行
        ↓ 遇到尚未完成的异步等待
当前 Task 暂停，事件循环运行其他就绪工作
        ↓ 等待条件完成
Task 从暂停处恢复，return 值成为 await 表达式的结果
```

学完后，你应当能解释：

- 协程函数、协程对象和最终返回值为什么是三个不同的东西；
- `await` 暂停的是谁，为什么它不等于“新建线程”；
- 为什么连续写两个 `await` 仍然是顺序执行；
- 为什么 `time.sleep()`、同步 HTTP 和长时间 CPU 计算会卡住事件循环；
- coroutine、Task 和 Future 有什么职责差异；
- 脚本为何通常只在最外层调用一次 `asyncio.run()`。

Task 的完整生命周期、取消、异常汇总和结构化并发会在第 20 章展开。本章先把协程与事件循环的基础机制讲清楚。

19.1 Java 工程师应该怎样建立第一版映射

Python 没有与 Java `async/await` 完全一一对应的语言模型。可以借用 `CompletableFuture`、线程池和 Netty EventLoop 帮助理解，但必须同时记住边界。

Java 中熟悉的概念	Python asyncio 中可类比的概念	关键差异
`ExecutorService.submit(callable)`	`asyncio.create_task(coro)`	`submit` 把工作交给线程池；Task 通常由事件循环在其线程中协作推进
`Future.get()` / `join()`	`await awaitable`	`get/join` 通常阻塞调用线程；等待未完成对象时，`await` 暂停当前 Task，把线程还给事件循环
`CompletableFuture<T>`	`asyncio.Future[T]` 或 Task	都保存未来的结果或异常；asyncio Future 通常绑定事件循环，也不是通用的线程间同步对象
单线程 Netty EventLoop	单线程 asyncio 事件循环	都怕阻塞回调；asyncio coroutine 还会通过 `await` 显式暴露协作暂停点
一次可恢复的状态机	coroutine 对象	coroutine 保存局部变量和暂停位置，但必须由 await、Task 或 runner 驱动

最容易误判的一点是“调用即启动”：

```
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(this::loadAnswer); // 通常已经提交执行
```

```
pending = load_answer()  # 只创建 coroutine 对象，函数体还没有执行
```

Python 的 coroutine 对象更像“一次尚未被驱动的可恢复调用”，而不是已经在线程池后台运行的 `CompletableFuture`。

19.2 asyncio 解决的是等待重叠，不是计算并行

先分清两组概念。

I/O 等待是请求已经发出，但结果还没准备好。例如网络响应尚未返回、socket 暂时没有字节可读、数据库仍在查询。此时当前工作没有有用的 Python 指令可执行。

CPU 执行是解释器正在解析 JSON、拼接 prompt、过滤数据或运行计算循环。只要代码持续占用事件循环线程，其他 Task 就无法在同一线程上运行。

概念	关心的问题	单线程 asyncio 能否做到
并发（concurrency）	多项工作能否在同一时间段交错推进	能；A 等 I/O 时可以推进 B
并行（parallelism）	多项计算能否在同一时刻真正执行	单个事件循环不能自动做到

因此，asyncio 最适合 **I/O 密集型**场景。它提升的通常是大量等待可以重叠时的吞吐量和响应性，不保证单个操作更快，也不会自动加速 CPU 密集型计算。

19.3 `async def`：调用函数时为什么没有执行函数体

普通 `def` 函数被调用后会立即进入函数体。`async def` 不同：它定义的是**协程函数**，调用后先得到一个**协程对象**。

```
import asyncio


async def fetch_answer() -> str:
    print("函数体开始")
    await asyncio.sleep(0)
    return "answer"


async def demo() -> None:
    pending = fetch_answer()  # 此时不会打印
    answer = await pending  # 从这里开始驱动函数体
    print(answer)  # answer


asyncio.run(demo())
```

这里必须分清三层：

名称	例子	含义
协程函数	`fetch_answer`	由 `async def` 定义的可调用对象
协程对象	`fetch_answer()` 的返回对象	保存这一次调用的参数、局部状态和执行位置
最终结果	`await pending` 得到的 `"answer"`	协程执行到 `return` 后交给调用方的普通值

README × | readme × | README × | README × | README × | README × | RE
'7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-5d769643fc71560c219e716e65.html
需求相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计费

`fetch_answer() -> str` 中的 `str` 描述的是**协程完成后的结果类型**。从运行时看，直接调用 `fetch_answer()` 得到的不是字符串，而是 coroutine 对象。

即使一个 `async def` 函数体里完全没有 `await`，调用它时仍然只会创建 coroutine 对象。区别只是：该对象一旦被驱动，函数体会连续执行到 `return`，中间没有主动暂停的机会。

> `async def` 中如果出现 `yield`，定义的是异步生成器，而不是本章讨论的普通 coroutine。异步生成器与 `async for` 会在后续章节学习。

19.4 `await`：暂停、恢复和返回值如何沿调用链传递

`await expression` 会先计算 `expression`。计算结果必须是 **awaitable（可等待对象）**。除支持顶层 `await` 的交互环境外，`await` 只能写在 `async def` 函数体内。asyncio 中最常见的 awaitable 有三类：

```
awaitable
├── coroutine：一次 async def 调用
├── Task：被事件循环调度的 coroutine，并保存其最终状态
└── Future：稍后才会有结果或异常的占位对象
```

业务代码最常直接创建 coroutine。Task 用于让工作获得独立调度机会，Future 则更常由框架、协议实现或底层 asyncio API 提供。

直接 await 子协程不会创建新线程或新 Task

```python
import asyncio


async def load_number() -> int:
    await asyncio.sleep(0)
    return 20


async def calculate() -> int:
    value = await load_number()
    return value + 1


print(asyncio.run(calculate()))  # 21
```

```
当前 Task 进入 calculate
  └── await load_number()
        └── 遇到尚未完成的异步等待
              当前 Task 连同整条调用链暂停
              事件循环获得控制权
        └── 等待完成后从原位置恢复，return 20
  └── await 表达式得到 20，calculate 继续执行并 return 21
```

局部变量和执行位置会保存在 coroutine 状态中，恢复时不会从函数开头重跑。子协程抛出的异常也会从对应的 `await` 位置向外传播，可以像同步调用链一样用 `try/except/finally` 处理；Task 间的异常管理留到下一章。

`await` 不保证每次都发生调度切换

如果 awaitable 已经完成， `await` 可以立即取得结果，当前 Task 不必暂停：

```python
import asyncio


async def read_completed() -> None:
    loop = asyncio.get_running_loop()
    completed = loop.create_future()
    completed.set_result("缓存命中")

    print("之前")
    result = await completed
    print("之后", result)


asyncio.run(read_completed())
```

近期报的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代

这里"之前"和"之后"可以连续出现。类似地，如果被直接 await 的 coroutine 内部没有遇到尚未完成的等待，它也可以一路执行到返回。

因此，准确规则不是"见到 `await` 就切换任务"，而是：

1. `await` 检查并推进 awaitable；
2. 若结果暂时无法取得，当前 Task 暂停；
3. 若结果已经就绪，`await` 可以立即返回；
4. 当前 Task 暂停后，事件循环才有机会选择其他就绪工作。

`await asyncio.sleep(0)` 有明确的协作暂停语义，适合教学实验或极少数主动让出执行权的场景。正常业务代码应等待真实的异步操作，而不是到处插入 `sleep(0)` 修补阻塞设计。

19.5 为什么连续两个 `await` 仍然是顺序执行

下面的完整小例子不会同时启动两项操作：

```
import asyncio


async def operation(name: str) -> str:
    await asyncio.sleep(0)
    return name


async def sequential() -> None:
    llm_result = await operation("LLM")
    rows = await operation("数据库")
    print(llm_result, rows)


asyncio.run(sequential())
```

执行顺序是：

```
创建并推进 call_llm 的 coroutine
  → 等待 LLM 时，当前 Task 可以暂停
  → LLM 完成，第一条语句得到结果
然后才调用 query_database
```

README | readme | README | README | README | README | README
h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-5d769643fc71560c219e716e65.html
求相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计费

执行顺序是：

```
创建并推进 call_llm 的 coroutine
  → 等待 LLM 时，当前 Task 可以暂停
  → LLM 完成，第一条语句得到结果
然后才调用 query_database
  → 创建并推进数据库 coroutine
  → 数据库完成，第二条语句得到结果
```

第二条语句必须等待第一条语句完成后才能执行，所以数据库 coroutine 甚至不会提前创建。

这并不一定是问题。如果数据库查询参数依赖 LLM 的输出，顺序正是业务约束。只有两项工作彼此独立时，才应该考虑并发调度：.

```python
async def concurrent() -> None:
    llm_task = asyncio.create_task(operation("LLM"))
    database_task = asyncio.create_task(operation("数据库"))

    llm_result = await llm_task
    rows = await database_task
    print(llm_result, rows)
```

`create_task()` 会把 coroutine 安排给事件循环，使两项工作都能在后续调度中向前推进。但 Task 必须有人保存引用、取得结果并处理异常；生产代码如何用 `TaskGroup` 管理这些责任是第 20 章的主题。

19.6 事件循环为什么最怕同步阻塞

在本章的单线程模型中，事件循环一次只推进一个 Task。Task 运行到尚未完成的异步等待时主动暂停，事件循环才能处理 I/O 完成事件、定时器和其他 Task。这叫**协作调度**。

把函数声明成 `async def`，不会改变函数体中普通调用的执行方式：

```python
import time


async def wrong() -> None:
    time.sleep(1)  # 阻塞事件循环所在的 OS 线程
    sum(i * i for i in range(10_000_000))  # 长计算持续占用事件循环线程
```

这与在 Netty EventLoop 中执行阻塞 JDBC 调用很相似：并不是只有当前请求变慢，而是同一循环负责的心跳、超时、I/O 回调和其他请求都会被推迟。

常见工作应这样选择：

工作类型	通常的处理方式	原因
原生异步 HTTP、数据库、MCP 客户端	直接 `await` 异步 API	等待期间能交还事件循环
无法替换的同步阻塞 I/O	`await asyncio.to_thread(func, ...)`	把阻塞调用移出事件循环线程
很短的 CPU 处理	直接执行	调度和传输也有成本
长时间 CPU 计算	进程、 可并行的原生实现或外部计算服务	单事件循环不提供计算并行

`asyncio.to_thread()` 主要用于兼容阻塞 I/O。它不会把任意 CPU 代码自动变快；受 GIL、原生扩展是否释放 GIL、数据传输和调度成本影响，CPU 并行需要单独设计。

19.7 coroutine、Task 和 Future 的生命周期不能混用

三者都可以被 `await`，但不是同一种对象。

对象	谁推进它	完成后能否再次 await	典型用途
coroutine	直接 await 它的调用链， 或交给 Task/runner	不能；同一个对象是一次性的	表示一次 `async def` 调用
Task	事件循环	可以观察同一个最终结果或异常	调度 coroutine， 并管理其完成状态
Future	由生产结果的一方调用 `set_result` / `set_exception`	可以观察同一个最终状态	框架和底层异步协议传递未来结果

同一个 coroutine 对象只能执行一次

```
pending = fetch_answer()
```

ers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-5d769643fc71560c219e716e65.html

的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

同一个 coroutine 对象只能执行一次

```
pending = fetch_answer()
first = await pending
second = await pending  # RuntimeError: cannot reuse already awaited coroutine
```

coroutine 对象保存的是一次调用的执行状态。完成后这条执行路径已经关闭，不能倒带。要再次执行，就重新调用协程函数：

```
first = await fetch_answer()
second = await fetch_answer()  # 新对象、新的一次调用
```

忘记 await，整个函数体可能一次都没运行

```
async def save_message() -> None:
    print("真的保存")


async def wrong() -> None:
    save_message()  # 只创建对象；既没有保存，也没有后台执行
```

未被驱动的 coroutine 最终被回收时，Python 通常会报告 `RuntimeWarning: coroutine ... was never awaited` 。这个警告只是问题线索，不能替代正确的生命周期管理。正确做法是按业务语义直接 `await` ，或把 coroutine 交给受管理的 Task。

示例代码为了演示错误，会对故意遗弃且尚未启动的 coroutine 调用 `close()` ，避免留下警告。`close()` 表示放弃这次调用，不是日常业务中的补救方案。

19.8 谁负责创建和关闭事件循环

普通 `.py` 脚本从同步世界启动，推荐在最外层建立一次清晰边界：

```
import asyncio


async def do_work() -> None:
    await asyncio.sleep(0)
```

rs/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-5d769643fc71560c219e716e65.html

的需求相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计费

19.8 谁负责创建和关闭事件循环

普通 `.py` 脚本从同步世界启动，推荐在最外层建立一次清晰边界：

```
import asyncio


async def do_work() -> None:
    await asyncio.sleep(0)


async def run_application() -> None:
    await do_work()


def main() -> None:
    asyncio.run(run_application())


if __name__ == "__main__":
    main()
```

`asyncio.run()` 会创建事件循环，驱动入口 awaitable，完成必要清理，然后关闭循环。业务库通常只暴露 `async def` API，把"怎样运行事件循环"的决定留给应用入口。

不要在 coroutine 内再次调用 `asyncio.run()`：

```
async def wrong() -> None:
    asyncio.run(other_coroutine())  # RuntimeError: 当前线程已有运行中的循环


async def correct() -> None:
    await other_coroutine()
```

Notebook、IPython、Web 框架和异步测试框架通常已经拥有事件循环。在支持顶层 `await` 的交互环境中，应使用 `await run_application()`；在框架中则遵循框架提供的生命周期入口。

19.9 Java 工程师最容易踩的迁移坑

19.9 Java 工程师最容易踩的迁移坑

误判	失败表现	正确做法
调用 `async def` 就会后台运行	函数体没有执行，出现 never-awaited 警告	直接 `await`，或创建并管理 Task
`await` 相当于 `Future.get()`	错把协作暂停理解为阻塞线程	关注 awaitable 是否未完成，以及 Task 是否把控制权交回循环
每个 `await` 都切换任务	对事件顺序作出错误假设	已完成 awaitable 可以立即返回；不要依赖猜测的调度顺序
连续两个 `await` 会并发	第二项工作始终在第一项完成后才开始	有依赖就保持顺序；独立工作才显式创建 Task
`async def` 会包装同步阻塞调用	整个事件循环卡住	换异步客户端，或用线程适配确实无法替换的阻塞 I/O
异步一定更快	单次请求没有变快，甚至多出调度成本	把 asyncio 用在大量可重叠等待上
coroutine 和 Future 都能重复等待	第二次 await 同一 coroutine 抛 `RuntimeError`	每次调用创建新 coroutine；不要混淆其生命周期
任意位置都能调用 `asyncio.run()`	运行中循环内抛 `RuntimeError`	只在同步程序最外层创建 runner

阅读一个陌生异步 API 时，可以按以下顺序判断：

1. 调用后得到普通值、coroutine、Task，还是 Future?
2. 它内部使用真正的异步 I/O，还是偷偷调用同步阻塞库?
3. 下一步是否依赖当前结果？依赖就顺序 await，独立才考虑并发。
4. 当前 coroutine 中是否存在 `time.sleep()`、同步网络调用或长 CPU 循环?
5. 如果创建了 Task，谁负责保存引用、取得结果、处理异常和清理?

19.10 运行本章示例

本章只使用 Python 标准库，不需要安装额外运行依赖。仓库根目录的 `pyproject.toml` 要求 Python 3.14 或更高版本。

在仓库根目录运行：

19.10 运行本章示例

本章只使用 Python 标准库，不需要安装额外运行依赖。仓库根目录的 `pyproject.toml` 要求 Python 3.14 或更高版本。

在仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/19_asyncio_coroutines/example.py
```

如果没有使用仓库内的 `.venv`，可将解释器路径替换为本机的 Python 3.14+。`-B` 只是不生成 `.pyc` 文件，不影响示例语义。

`example.py` 包含六组可独立观察的实验。运行时重点看这些事实：

1. 创建 coroutine 对象后，函数体事件尚未出现；await 后才得到最终结果。
2. 三层直接 await 按"由外向内进入、由内向外返回"的顺序工作。
3. await 已完成 Future 时，旁路 Task 不会强行插入两条相邻事件之间。
4. Agent 流程中，数据库替身在 LLM 替身完成后才开始。
5. 协作等待期间 heartbeat 能推进，`time.sleep()` 期间 heartbeat 完全停住。
6. 重复 await、忘记 await 和嵌套 `asyncio.run()` 都会以受控方式展示并清理。

`example.py` 中的 `asyncio.sleep(0)` 只是可控暂停点，不模拟 DNS、TLS、网络超时、重试、连接池或真实服务延迟。它能证明控制流，不代表真实客户端一定是非阻塞实现。

当前 `java_to_python/run_all.py` 只包含前 10 章，因此第 19 章需要使用上面的命令单独运行。可选的静态检查命令是：

```
.venv/bin/ruff check java_to_python/chapters/19_asyncio_coroutines
.venv/bin/ruff format --check java_to_python/chapters/19_asyncio_coroutines
.venv/bin/pyright java_to_python/chapters/19_asyncio_coroutines/example.py
```

19.11 动手练习

练习 1：先预测，再验证 coroutine 生命周期

实现 `async def normalize_prompt(text: str) -> str`：先记录"开始"，执行一次 `await asyncio.sleep(0)`，再记录"恢复"，最后返回去除首尾空白的字符串。

19.11 动手练习

练习 1：先预测，再验证 coroutine 生命周期

实现 `async def normalize_prompt(text: str) -> str`：先记录“开始”，执行一次 `await asyncio.sleep(0)`，再记录“恢复”，最后返回去除首尾空白的字符串。

验证调用后、await 前事件列表为空；await 后结果为 `"hello asyncio"`，事件顺序为“开始、恢复”。

练习 2：判断顺序还是并发

实现 `fetch_profile(user_id)` 和 `build_reply(profile)` 两个本地 coroutine。先让第二步依赖第一步的返回值，记录完整事件顺序；再设计两个彼此独立的 coroutine，预测顺序 await 与创建 Task 后的差异。

重点不是比较毫秒数，而是用事件记录证明“谁先创建、谁先开始、谁在何处恢复”。

练习 3：证明阻塞会饿死其他 Task

创建一个不断递增计数并 `await asyncio.sleep(0)` 的 heartbeat Task，分别执行一次协作等待和一次很短的 `time.sleep()`。用等待前后的计数断言：协作等待期间 heartbeat 前进，阻塞调用期间计数不变。

记得在 `finally` 中通知 heartbeat 退出并 await 它，避免遗留后台 Task。

19.12 本章小结

- `async def` 定义协程函数；调用只创建一次性的 coroutine 对象。
- coroutine 必须被 await、包装成 Task，或交给 `asyncio.run()`，函数体才会执行。
- `await` 等待未完成对象时暂停当前 Task，不会自动创建线程。
- 直接 await 子 coroutine 会沿同一 Task 的调用链进入和返回。
- `await` 遇到已完成对象时可以立即返回，不保证发生调度切换。
- 连续两条 await 语句默认顺序执行；并发需要显式调度独立工作。
- 同步阻塞 I/O 和长 CPU 代码会占住事件循环线程，`async def` 不会改变这一点。
- coroutine 是一次性执行状态；Task 和 Future 保存完成状态，复用规则不同。
- `asyncio.run()` 通常只出现在同步程序的最外层入口。

如果你能仅凭事件顺序解释“对象何时创建、函数体何时开始、哪条调用链暂停、由谁恢复、结果如何返回”，就已经掌握了阅读 asyncio 业务代码最重要的基础。

```python
"""第 19 章：asyncio coroutine、await 与事件循环。

示例只使用本地、可控的替身来暴露暂停与恢复顺序，不访问真实 LLM、数据库或网络。
每段都用事件列表做运行时断言，因此观察结果不依赖精确耗时或偶然的调度顺序。
"""

import asyncio
import inspect
import time

# asyncio 提供事件循环、协程调度工具以及 Event/Future/Task 等异步原语。
# inspect 在这里不参与调度，只用来辨认“协程函数 / 协程对象”并读取对象状态。
# time.sleep 则故意充当同步阻塞反例，用来和 asyncio 的协作等待形成对照。


# 这两个同步辅助函数只负责展示和收集事件。它们内部没有 await，调用时会立即执行完毕；
# `title: str`、`-> None` 等类型提示主要服务读者和静态检查器，不会像 Java 类型那样在运行时强制校验。
def print_section(title: str) -> None:
    # f-string 会先计算花括号里的 title，再嵌入字符串；开头的 \n 让各实验分段显示。
    print(f"\n=== {title} ===")


def record(events: list[str], message: str) -> None:
    """同时保存和打印事件，让控制流既可观察又可断言。"""
    # events 是调用方传入的同一个可变 list。append 会原地修改它，因此调用方随后能看到新事件；
    # 这不同于重新给局部变量赋一个新列表。保留事件后再打印，既方便人读，也方便后面精确断言顺序。
    events.append(message)
    print(f"  事件 | {message}")


# `async def` 定义的是协程函数。调用它只创建 coroutine 对象，不会立即执行下面的函数体；
# 当对象被 await、被包装成 Task，或交给 asyncio.run 后，函数体才会被推进。
# 返回注解 `-> str` 描述协程最终完成时交出的普通字符串，而不是调用瞬间的返回对象类型。
async def lifecycle_coroutine(events: list[str]) -> str:
    """用一个明确会暂停的检查点展示协程体何时启动、恢复和返回。"""
    record(events, "函数体开始")
    # sleep(0) 不模拟真实延迟，只提供一个保证会暂停当前 Task 的协作检查点。
```

```python
async def lifecycle_coroutine(events: list[str]) -> str:
    """用一个明确会暂停的检查点展示协程体何时启动、恢复和返回。"""
    record(events, "函数体开始")
    # sleep(0) 不模拟真实延迟，只提供一个保证会暂停当前 Task 的协作检查点。
    # `await` 保存当前协程的执行位置并把执行权交还事件循环；以后再次获得运行机会时，
    # 从下一行继续，而不是从函数开头重新执行。
    await asyncio.sleep(0)
    record(events, "恢复")
    record(events, "返回")
    return "生命周期结果"


# 第一组实验把“函数”“一次调用产生的对象”“对象完成后的结果”明确分开。
async def demonstrate_coroutine_lifecycle() -> None:
    print_section("1. async def、协程函数、协程对象与最终结果")
    # `list[str]` 表示期望元素都是字符串。运行时它仍是普通 list，类型参数不会自动检查 append 的值。
    events: list[str] = []

    # 这次调用把参数 events 保存进新 coroutine 对象 pending，但 lifecycle_coroutine 的函数体尚未运行。
    # 因而“创建前”和“创建后”会相邻出现，列表中暂时不会有“函数体开始”。
    record(events, "创建前")
    pending: CoroutineType[Any, Any, str] = lifecycle_coroutine(events)
    record(events, "创建后")

    # 第一个检查针对函数本身，第二个检查针对调用得到的 pending 对象，二者不是同一个概念。
    # getcoroutinestate 返回枚举式状态字符串；尚未被推进的原生协程处于 CORO_CREATED。
    print("  lifecycle_coroutine 是协程函数:", inspect.iscoroutinefunction(lifecycle_coroutine))
    print("  调用所得对象是 coroutine:", inspect.iscoroutine(pending))
    print("  刚创建时状态:", inspect.getcoroutinestate(pending))
    print("  函数体是否已经开始:", "函数体开始" in events)
    # `assert` 在条件为假时抛 AssertionError。本例用它把教学结论变成可执行的验证，
    # 而不只是依赖肉眼查看打印文本。
    assert events == ["创建前", "创建后"]
    assert inspect.getcoroutinestate(pending) == inspect.CORO_CREATED

    # await 开始推进 pending。它在内部 sleep(0) 处暂停一次，恢复后执行到 return；
```

```python
async def demonstrate_coroutine_lifecycle() -> None:

    # await 开始推进 pending。它在内部 sleep(0) 处暂停一次，恢复后执行到 return；
    # 整个 await 表达式的值就是 return 的字符串，并被绑定到 result。
    result: str = await pending

    # 已执行到 return 的原生 coroutine 会关闭，不能把同一个对象倒带后再运行一次。
    # 第 6 组实验会专门演示重复 await 时的 RuntimeError。
    print("  await 表达式得到最终结果:", result)
    print("  完成后的对象状态:", inspect.getcoroutinestate(pending))
    assert events == ["创建前", "创建后", "函数体开始", "恢复", "返回"]
    assert result == "生命周期结果"
    assert inspect.getcoroutinestate(pending) == inspect.CORO_CLOSED


# 接下来的三个协程组成一条嵌套调用链。直接 await 子协程不会为每一层创建新线程或新 Task；
# 当前 Task 会依次进入外层、中层和内层，直到最深处遇到真正会暂停的 awaitable。
async def load_leaf(events: list[str]) -> int:
    record(events, "内层：开始")
    # 整条调用链在这里暂停；事件循环获得控制权，之后再从本行 await 的后面恢复。
    await asyncio.sleep(0)
    record(events, "内层：等待结束后恢复")
    return 20


async def transform_middle(events: list[str]) -> int:
    record(events, "中层：开始并 await 内层")
    # 调用 load_leaf 创建内层 coroutine，await 推进它并等待其 return；value 最终得到整数 20。
    value: int = await load_leaf(events)
    record(events, f"中层：拿到 {value} 后恢复")
    return value + 1


async def serve_outer(events: list[str]) -> int:
    record(events, "外层：开始并 await 中层")
    # 中层完成后把 21 作为 await 表达式的值传回这里，外层才会继续计算。
    value: int = await transform_middle(events)
```

```python

async def serve_outer(events: list[str]) -> int:
    record(events, "外层：开始并 await 中层")
    # 中层完成后把 21 作为 await 表达式的值传回这里，外层才会继续计算。
    value: int = await transform_middle(events)
    record(events, f"外层：拿到 {value} 后恢复")
    return value * 2


# 第二组实验从调用链最外层进入，用一份完整事件序列验证“由外向内进入、由内向外返回”。
async def demonstrate_await_chain() -> None:
    print_section("2. await 调用链怎样暂停与恢复")
    events: list[str] = []
    # result 要等 serve_outer 连同全部子协程完成后才会得到最终整数 42。
    result: int = await serve_outer(events)

    # expected 是预先写下的精确顺序。把实际列表与它整体比较，可以捕捉缺失、重复或次序错误。
    expected: list[str] = [
        "外层：开始并 await 中层",
        "中层：开始并 await 内层",
        "内层：开始",
        "内层：等待结束后恢复",
        "中层：拿到 20 后恢复",
        "外层：拿到 21 后恢复",
    ]
    print("  最终结果:", result)
    assert events == expected
    assert result == 42


# 这个协程没有任何 await：它一旦被 Task 真正推进，就会连续执行到隐式返回 None。
# “没有 await”并不改变其协程函数身份，只说明函数体内部没有主动暂停点。
async def peer_coroutine(events: list[str]) -> None:
    record(events, "旁路 Task：得到运行机会")


```

```
134
135  # 第三组实验反驳“代码里出现 await，就一定先切换到其他 Task”这一误解。
136  async def demonstrate_immediate_await() -> None:
137      print_section("3. 写了 await 不等于一定切换调度")
138      events: list[str] = []
139
140      # 调用 peer_coroutine 先产生 coroutine 对象，create_task 再把它交给事件循环管理。
141      # 这个 Task 只是观察器；name 便于调试，eager_start=False 明确要求它先排队而不立刻启动。
142      # 完整 Task 生命周期留到下一章，本例只用它判断旁路工作是否获得过运行机会。
143      peer_task: Task[None] = asyncio.create_task(
144          peer_coroutine(events),
145          name="immediate-await-observer",
146          eager_start=False,
147      )
148      # 当前代码已在 asyncio.run 建立的事件循环中，所以 get_running_loop 能取得正在运行的 loop。
149      # Future 可理解为“将来结果的占位对象”；泛型参数 str 是对其最终结果类型的静态说明。
150      loop: AbstractEventLoop = asyncio.get_running_loop()
151      completed: asyncio.Future[str] = loop.create_future()
152      # set_result 立即把 Future 标记为已完成，并保存字符串结果；这里没有真实 I/O 要等待。
153      completed.set_result("缓存命中")
154
155      record(events, "await 已完成 Future: 之前")
156      # completed 已经就绪，await 可同步取出其结果，因此当前 Task 无需暂停，旁路 Task 还不能插进来。
157      result: str = await completed
158      record(events, "await 已完成 Future: 之后")
159
160      # Future 已经有结果，await 直接取得它；当前代码没有为旁路 Task 让出运行机会。
161      assert events == ["await 已完成 Future: 之前", "await 已完成 Future: 之后"]
162      # peer_task 尚未完成，await 它时当前协程需要等待；事件循环于是推进旁路协程，直到其结束。
163      await peer_task
164
165      # `events[-1]` 用负索引读取列表最后一个元素，验证旁路事件是在前两条事件之后才出现。
166      print("  已完成 Future 的结果:", result)
167      print("  旁路 Task 只在后续真正暂停时才运行")
168      assert events[-1] == "旁路 Task: 得到运行机会"
169
170
```

```python

# 下面两个函数是确定性的本地替身。它们保留"发起工作 -> 等待 -> 结果就绪"的形状，
# 只用于讲解控制流，不代表真实 LLM 或数据库客户端的网络、超时、重试等行为。
async def simulated_llm(prompt: str, events: list[str]) -> str:
    """可控替身：只模拟一次能协作暂停的等待，不模拟真实 LLM 协议。"""
    # f-string 中的 `!r` 使用 repr(prompt)，所以输出会带引号，便于看清字符串边界。
    record(events, f"LLM: 开始处理 {prompt!r}")
    await asyncio.sleep(0)
    record(events, "LLM: 响应就绪")
    return "需要查询订单 42"


async def simulated_database(query: str, events: list[str]) -> list[str]:
    """可控替身：不建立连接，只展示数据库步骤何时才开始。"""
    record(events, f"数据库：开始执行 {query!r}")
    await asyncio.sleep(0)
    record(events, "数据库：结果就绪")
    # 返回单元素 list，模拟数据库的多行结果；调用方稍后用 rows[0] 读取第一条记录。
    return ["订单 42: 已发货"]


# 这个 Agent 流程有真实的数据依赖：必须先拿到模型判断，才进入后续数据库步骤。
# 连续写两个 await 只允许同一调用链在等待时让出事件循环，并不会让两个操作并发启动。
async def handle_agent_request(events: list[str]) -> str:
    record(events, "Agent: 收到请求")
    # 右侧 coroutine 完成前不会执行下一条语句；intent 接收 simulated_llm 的最终字符串结果。
    intent: str = await simulated_llm("订单到哪里了？", events)
    record(events, f"Agent: 拿到模型结果 {intent!r}")

    # 只有第一段 await 完成后，Python 才调用 simulated_database 创建并推进第二个 coroutine。
    rows: list[str] = await simulated_database("select order 42", events)
    # len(rows) 读取结果条数；rows[0] 则取第一条记录作为整个 Agent 协程的最终返回值。
    record(events, f"Agent: 拿到 {len(rows)} 条数据库记录")
    return rows[0]


```

```python

# 第四组实验不用计时，而是比较事件在列表中的位置，证明两个替身严格顺序执行。
async def demonstrate_sequential_agent_io() -> None:
    print_section("4. Agent 场景：连续 await 仍是顺序执行")
    events: list[str] = []
    answer: str = await handle_agent_request(events)

    # list.index 返回完全匹配元素第一次出现的下标。
    llm_finished: int = events.index("LLM：响应就绪")
    # 生成器表达式逐个产生符合条件的下标；next 只取第一个。
    # enumerate(events) 每轮给出 `(下标，消息)` 二元组，再解包给 index 和 message。
    database_started: int = next(
        index for index, message in enumerate(events) if message.startswith("数据库：开始执行")
    )
    print("  最终回答:", answer)
    print("  可观察结论：数据库开始位置晚于 LLM 完成位置")
    assert llm_finished < database_started
    assert answer == "订单 42：已发货"


# heartbeat 是并发观察器：每次事件循环调度到它，就把共享计数加一并发出一次 Event 信号。
async def heartbeat(
    stop: asyncio.Event,
    next_pulse: asyncio.Event,
    pulse_count: list[int],
    events: list[str],
) -> None:
    """最小观察器：每次得到事件循环执行机会就发出一个心跳。"""
    # try/finally 保证无论正常退出还是外部取消，都能记录“已停止”；对应 Java 的 finally 清理语义。
    try:
        # Event 是电平式标志，不是自动累加消息的队列。只要 stop 尚未被 set，就继续产生心跳。
        while not stop.is_set():
            # int 本身不可变，因此用单元素 list 作为共享可变容器；原地改索引后，调用方能看到新计数。
            pulse_count[0] += 1
            record(events, f"heartbeat：第 {pulse_count[0]} 次")
            # set 令所有等待 next_pulse 的协程具备恢复条件；它不会在这里阻塞 heartbeat。
            next_pulse.set()
```

```python
# heartbeat 是并发观察器：每次事件循环调度到它，就把共享计数加一并发出一次 Event 信号。
async def heartbeat(
    stop: asyncio.Event,
    next_pulse: asyncio.Event,
    pulse_count: list[int],
    events: list[str],
) -> None:
    """最小观察器：每次得到事件循环执行机会就发出一个心跳。"""
    # try/finally 保证无论正常退出还是外部取消，都能记录“已停止”；对应 Java 的 finally 清理语义。
    try:
        # Event 是电平式标志，不是自动累加消息的队列。只要 stop 尚未被 set，就继续产生心跳。
        while not stop.is_set():
            # int 本身不可变，因此用单元素 list 作为共享可变容器；原地改索引后，调用方能看到新计数。
            pulse_count[0] += 1
            record(events, f"heartbeat: 第 {pulse_count[0]} 次")
            # set 令所有等待 next_pulse 的协程具备恢复条件；它不会在这里阻塞 heartbeat。
            next_pulse.set()
            # 主动让出一次执行权，否则这个无 I/O 的 while 循环会一直占住事件循环线程。
            await asyncio.sleep(0)
    finally:
        record(events, "heartbeat: 已停止")


# 当前协程通过等待“下一次心跳”来证明：自己暂停期间，旁路 Task 的确向前推进了。
async def cooperative_wait(next_pulse: asyncio.Event, events: list[str]) -> None:
    """等待明确的下一次心跳，证明当前协程确实把执行权交回了事件循环。"""
    record(events, "协作等待：开始")
    # 先 clear 丢弃旧的已触发状态，确保下面 wait 等到的是本次实验期间产生的新心跳。
    next_pulse.clear()
    # 标志尚未 set 时，wait 会暂停当前 Task；heartbeat 随后获得运行机会并调用 set。
    await next_pulse.wait()
    record(events, "协作等待：看到等待期间的新心跳后恢复")


# 反例虽然写成 async def，函数体中的普通同步调用仍按同步方式执行。
async def blocking_wait(events: list[str]) -> None:
```

```
258
259 # 反例虽然写成 async def，函数体中的普通同步调用仍按同步方式执行。
260 async def blocking_wait(events: list[str]) -> None:
261     """教学反例：async def 不会改变 time.sleep 的阻塞性质。"""
262     record(events, "阻塞等待：开始")
263     # 时长只用于让现象便于人眼感受；断言只检查事件顺序，不比较耗时。
264     # time.sleep 不返回 awaitable，也不会把控制权还给事件循环；同一线程上的 heartbeat 因此无法运行。
265     time.sleep(0.05)  # noqa: ASYNC251 - 本函数专门演示应被检查器阻止的写法。
266     record(events, "阻塞等待：结束")
267
268
269 # 第五组实验用共享计数和 Event 构造确定性证据，避免用脆弱的耗时大小猜测是否发生调度。
270 async def demonstrate_blocking_event_loop() -> None:
271     print_section("5. heartbeat：协作等待与阻塞事件循环")
272     events: list[str] = []
273     # 用 list 包装计数，heartbeat 对 pulse_count[0] 的原地修改可被本协程直接观察。
274     pulse_count: list[int] = [0]
275     # 两个 Event 分工不同：stop 通知观察器退出，next_pulse 通知当前协程“新心跳已发生”。
276     stop = asyncio.Event()
277     next_pulse = asyncio.Event()
278     # 保存 Task 引用，才能在 finally 中负责它的完整退出；后台工作不应被创建后就遗忘。
279     heartbeat_task: Task[None] = asyncio.create_task(
280         heartbeat(stop, next_pulse, pulse_count, events),
281         name="heartbeat-observer",
282         eager_start=False,
283     )
284
285     try:
286         # 先等到观察器确实启动，后面的结论不依赖谁先被调度。
287         await next_pulse.wait()
288
289         # 记录协作等待前后的快照。cooperative_wait 只有看到一次新 pulse 才返回，所以计数必须增加。
290         before_cooperative: int = pulse_count[0]
291         await cooperative_wait(next_pulse, events)
292         after_cooperative: int = pulse_count[0]
293
294         # 清除上一步留下的触发状态，再记录同步阻塞前后的快照。
```

```
270  async def demonstrate_blocking_event_loop() -> None:
294          # 清除上一步留下的触发状态，再记录同步阻塞前后的快照。
295          # blocking_wait 内没有真正的异步暂停点，因此其执行期间计数必须保持不变。
296          next_pulse.clear()
297          before_blocking: int = pulse_count[0]
298          await blocking_wait(events)
299          after_blocking: int = pulse_count[0]
300
301          # 差值只关心“是否推进”，不要求事件循环恰好产生几次心跳，也不依赖机器速度。
302          print("  协作等待期间新增心跳:", after_cooperative - before_cooperative)
303          print("  阻塞等待期间新增心跳:", after_blocking - before_blocking)
304          assert after_cooperative > before_cooperative
305          assert after_blocking == before_blocking
306          assert not next_pulse.is_set()
307      finally:
308          # 不遗留后台 Task；即使断言失败，也通知观察器退出并等待清理完成。
309          # set 只发退出信号，await heartbeat_task 才真正等到观察器运行 finally 并结束。
310          stop.set()
311          await heartbeat_task
312
313
314  # 第六组实验会先正常完成这个协程，再故意重复等待同一个对象。
315  async def returns_once(events: list[str]) -> str:
316      record(events, "一次性协程：函数体执行")
317      return "完成"
318
319
320  # 该函数只作为嵌套 asyncio.run 的参数；若外层事件循环正确拒绝嵌套，它的函数体就不会执行。
321  async def nested_run_target() -> None:
322      print("这行不应执行")
323
324
325  # 最后一组把三个常见错误放进受控的 try/except/finally 中，既观察异常，也清理故意创建的对象。
326  async def demonstrate_controlled_errors() -> None:
327      print_section("6. 一次性对象、忘记 await 与嵌套 asyncio.run")
328
```

```python
# 最后一组把三个常见错误放进受控的 try/except/finally 中，既观察异常，也清理故意创建的对象。
async def demonstrate_controlled_errors() -> None:
    print_section("6. 一次性对象、忘记 await 与嵌套 asyncio.run")

    reuse_events: list[str] = []
    # one_object 代表 returns_once 的这一次具体调用；第一次 await 会把它一直推进到 return。
    one_object: CoroutineType[Any, Any, str] = returns_once(reuse_events)
    first_result: str = await one_object
    print("  第一次 await 得到:", first_result)
    try:
        # 原生 coroutine 对象保存的是一次性执行状态，完成后已关闭；第二次 await 必须失败。
        await one_object
    except RuntimeError as error:
        # `as error` 把捕获到的异常对象绑定给局部变量；type(error).__name__ 读取异常类名。
        print("  第二次 await 同一对象:", type(error).__name__, str(error))
    else:
        # else 只在 try 没抛异常时执行；走到这里反而说明预期规则没有成立，因此主动令实验失败。
        raise AssertionError("已完成的原生协程对象不应允许第二次 await")
    assert reuse_events == ["一次性协程：函数体执行"]

    forgotten_events: list[str] = []
    # 这里只调用而不 await：forgotten 被创建，但 returns_once 的函数体事件仍然是 0 条。
    forgotten: CoroutineType[Any, Any, str] = returns_once(forgotten_events)
    print("  只调用不 await 后，函数体事件数:", len(forgotten_events))
    print("  未启动对象状态:", inspect.getcoroutinestate(forgotten))
    # 教学中故意创建了错误对象，所以显式 close，避免正常入口留下 RuntimeWarning。
    # close 表示放弃这次调用，使对象直接进入 CORO_CLOSED；正常业务应 await 或受控调度它，
    # 而不是靠 close 补做原本期望发生的工作。
    forgotten.close()
    print("  显式 close 后状态:", inspect.getcoroutinestate(forgotten))
    assert forgotten_events == []
    assert inspect.getcoroutinestate(forgotten) == inspect.CORO_CLOSED

    # 先创建对象，是为了把它传给 asyncio.run；但当前函数本身已运行在外层事件循环中。
    nested: CoroutineType[Any, Any, None] = nested_run_target()
    try:
        # asyncio.run 负责创建并独占一个顶层事件循环，同一线程已有运行中循环时禁止再次调用。
```

```python
async def demonstrate_controlled_errors() -> None:
    nested: Coroutine[Any, Any, None] = nested_run_target()
    try:
        # asyncio.run 负责创建并独占一个顶层事件循环，同一线程已有运行中循环时禁止再次调用。
        asyncio.run(nested)
    except RuntimeError as error:
        print("  运行中事件循环内再调用 asyncio.run:", type(error).__name__, str(error))
    else:
        raise AssertionError("同一线程已有事件循环时，asyncio.run 应拒绝嵌套调用")
    finally:
        # asyncio.run 在接管对象前就失败；调用方仍负责关闭刚创建的协程对象。
        nested.close()


# run_examples 是全章唯一的异步总入口。六个 await 按源码顺序逐组完成，并不让实验彼此并发。
async def run_examples() -> None:
    await demonstrate_coroutine_lifecycle()
    await demonstrate_await_chain()
    await demonstrate_immediate_await()
    await demonstrate_sequential_agent_io()
    await demonstrate_blocking_event_loop()
    await demonstrate_controlled_errors()
    print("\n所有第 19 章示例与运行时断言均已完成。")


# 普通脚本入口不能直接写顶层 await，因此用一次 asyncio.run 建立事件循环并推进顶层 coroutine。
def main() -> None:
    """同步脚本入口只调用一次 asyncio.run，由它创建并关闭事件循环。"""
    asyncio.run(run_examples())


# 导入本文件时，Python 只定义上面的函数，不自动运行实验；直接执行本文件时 __name__ 才是
# "__main__"，于是调用同步 main。实际顺序是 main -> asyncio.run -> run_examples -> 六组实验。
if __name__ == "__main__":
    main()
```